

CS 3210 Lecture Notes: Algorithm Design and Analysis

Aarush Ram Anandh

May 2, 2026

Contents

1	Introduction: What is Algorithm Analysis?	3
2	Lecture 1: Computational Models	3
3	Lecture 2	4
4	Lecture 3	7
5	Lecture 4: Sorting and Radix Algorithms	10
6	Lecture 5: Maximal Points and Geometric Algorithms	14
7	Lecture 6: Divide-and-Conquer and Polynomial Multiplication	17
8	Lecture 7: Order Statistics and Selection	22
9	Lecture 8	26
10	Lecture 9	27
11	Lecture 10	28
12	Lecture 11	29
13	Lecture 12	29
14	Lecture 13	30
15	Lecture 14	30
16	Lecture 15	31
17	Lecture 16	31
18	Lecture 17	32
19	Lecture 18	33
20	Lecture 19	33

21 Lecture 20	34
22 Lecture 21	34
23 Lecture 22	35
24 Lecture 23	36
25 Lecture 24	36
26 Lecture 25	37

1 Introduction: What is Algorithm Analysis?

Why Study Algorithms?

An algorithm is a step-by-step procedure to solve a computational problem. Algorithm analysis studies how efficiently algorithms use computational resources—primarily time (number of operations) and space (memory). We measure efficiency as a function of input size, typically denoted n .

Asymptotic Notation

We use Big-O notation to describe algorithm complexity:

- $O(f(n))$: the algorithm runs in at most $f(n)$ time in the worst case.
- $\Omega(f(n))$: the algorithm takes at least $f(n)$ time.
- $\Theta(f(n))$: the algorithm takes exactly $f(n)$ time (both upper and lower bounds).

Example: Linear search takes $\Theta(n)$ time—we must examine up to n elements in the worst case.

2 Lecture 1: Computational Models

The RAM Model

The Random Access Memory (RAM) model is the standard computational model in algorithm analysis. It assumes:

1. A processor can perform simple operations (arithmetic, logic, memory access) in constant time $O(1)$.
2. Memory is randomly accessible—reading or writing any memory cell takes $O(1)$ time.
3. All integers fit in a single machine word.

The RAM model is useful for analyzing most algorithms. However, it has limitations when dealing with very large numbers.

Why the Unit-Cost RAM Model Can Mislead

The RAM model treats all arithmetic operations as taking unit cost, regardless of operand size. This is unrealistic for large numbers. Consider:

```
x = 2
for i = 1 to n:
    x = x * x
```

After iteration 1: $x = 4 = 2^2$ After iteration 2: $x = 16 = 2^4$ After iteration 3: $x = 256 = 2^8$
After iteration n : $x = 2^{2^n}$

Under the RAM model, this appears to take $O(n)$ time. But the actual value grows to astronomical sizes! For instance, $2^{2^{10}} \approx 10^{308}$ —larger than the number of atoms in the universe.

Bit Complexity

To measure complexity accurately when numbers grow large, we use *bit complexity*: count the number of bit operations required.

If an integer has b bits, its value is at most 2^b . In the loop above, after n iterations:

$$x = 2^{2^n} \quad \text{requires} \quad \lfloor \log_2(2^{2^n}) \rfloor + 1 = 2^n + 1 \text{ bits}$$

The number of bits needed grows exponentially! Multiplying two b -bit numbers using grade-school multiplication costs $O(b^2)$ bit operations.

Thus, the true complexity of the loop is:

$$\text{Cost} = \sum_{i=0}^{n-1} O((2^i)^2) = \sum_{i=0}^{n-1} O(2^{2i}) = O(2^{2n})$$

This is exponential, not linear!

Key Lesson: Measure by Input Size

The critical principle: **time and space complexity must be measured as a function of the input size in bits**, not the value of numbers.

For an algorithm to be considered efficient, its running time should be polynomial in the number of input bits. Algorithms with exponential bit complexity are generally impractical.

3 Lecture 2

Review: Unit-cost RAM vs. Bit Complexity

The unit-cost RAM model treats every arithmetic operation as $O(1)$, which can be misleading when numbers grow very large. We illustrated this with the squaring loop.

Exponentiation by Squaring: A Concrete Example

Problem

Compute x^n where x is a real number and n is a positive integer. This is crucial in cryptography, where we often need to compute $a^b \pmod{m}$ for very large values.

Naive Approach

```
def naive_power(x, n):
    result = 1
    for i in range(n):
        result = result * x
    return result
```

This requires $n - 1$ multiplications. If x has b bits, then x^n has roughly nb bits. Each multiplication of b -bit numbers costs $O(b^2)$ time, so the total time is:

$$\text{Cost} = (n - 1) \cdot O((nb)^2) = O(n^3b^2)$$

This is extremely slow for large n . For $n = 1000$ and $b = 64$, we perform 1000 multiplications of increasingly large numbers.

Exponentiation by Squaring (Fast Exponentiation)

Key insight: use the binary representation of n .

```
def fast_power(x, n):
    if n == 0:
        return 1
    if n is even:
        half = fast_power(x, n // 2)
        return half * half
    else:
        return x * fast_power(x, n - 1)
```

The recurrence relation:

$$x^n = \begin{cases} (x^{n/2})^2, & n \text{ even} \\ x \cdot x^{n-1}, & n \text{ odd} \end{cases}$$

Alternatively, if $n = \sum_{i=0}^k b_i 2^i$ (binary representation), then:

$$x^n = \prod_{i=0}^k x^{b_i 2^i} = \prod_{i: b_i=1} x^{2^i}$$

Analysis

The recursion depth is $O(\log n)$ because we halve n at each step. Thus we perform only $O(\log n)$ multiplications. The total bit complexity is:

$$O(\log n) \text{ multiplications of } O(nb) \text{ bits each} = O(\log n \cdot (nb)^2) = O(n^2 b^2 \log n)$$

This is dramatically faster than $O(n^3 b^2)$ for large n . For $n = 1000$, we reduce from ~ 1000 multiplications to ~ 10 multiplications.

Example

To compute 2^{13} :

- Binary: $13 = 1101_2 = 8 + 4 + 1$
- $2^1 = 2$
- $2^2 = 2^1 \cdot 2^1 = 4$
- $2^4 = 2^2 \cdot 2^2 = 16$
- $2^8 = 2^4 \cdot 2^4 = 256$
- $2^{13} = 2^8 \cdot 2^4 \cdot 2^1 = 256 \cdot 16 \cdot 2 = 8192$

Only 6 multiplications instead of 12!

Example: Trial Division for Factoring

Problem

Given an integer m , determine if it is prime. If composite, find at least one factor.

Algorithm

```
def trial_division(m):
    for k in range(2, int(m**0.5) + 1):
        if m % k == 0:
            return k # k is a factor
    return None # m is prime
```

Complexity Analysis

In the worst case (when m is prime), we check all integers up to $\sqrt{m}$. This is $O(\sqrt{m})$ trial divisions.

But what is the input size? If m is represented as a binary string, it has $b = \log_2 m$ bits. Thus $m \leq 2^b$ and $\sqrt{m} \leq 2^{b/2}$.

The number of divisions is $\Theta(2^{b/2})$, which is *exponential in the input size*. This algorithm is impractical for large primes.

For example:

- 64-bit integer: up to $2^{32} \approx 4$ billion trial divisions—feasible but slow.
- 128-bit integer: up to 2^{64} trial divisions—infeasible.
- 1024-bit integer (common in cryptography): up to 2^{512} trial divisions—impossible.

This demonstrates why simple algorithms fail for cryptographic operations and why more sophisticated factoring algorithms (Pollard’s rho, quadratic sieve, etc.) are necessary.

Unit-cost RAM vs. Bit Complexity (Revisited)

The unit-cost RAM model treats every arithmetic operation as $O(1)$, which can be misleading when numbers grow very large. Consider the loop below starting with $x = 2$

```
for i = 1 to n:  
  x = x * x  
print x
```

Each iteration squares x . By induction, after n iterations,

$$x = 2^{2^n}.$$

In the unit-cost RAM model this appears to take $O(n)$ steps, but the value of x quickly becomes astronomical, so arithmetic is no longer constant time.

Bit length and size of numbers

If an integer is represented with b bits, its magnitude is at most 2^b . The number of bits in 2^{2^n} is

$$\lfloor \log_2(2^{2^n}) \rfloor + 1 = 2^n + 1.$$

Thus the intermediate values in the loop grow exponentially in n . The cost of multiplying two b -bit numbers using grade-school multiplication is $O(b^2)$ (or more generally some function $M(b)$ depending on the multiplication algorithm).

Measuring time by input size

Time and space should be measured in terms of the input size, i.e., the number of input bits. The unit-cost RAM model does not reflect the true running time when arithmetic costs depend on operand size.

Example: trial division for factoring

Given an integer m , the simplest factoring algorithm is:

```

for k = 2 to floor(sqrt(m)):
    if k divides m:
        report k as a factor
if no factor found: m is prime

```

This performs $O(\sqrt{m})$ divisibility tests. If m has b bits, then $m \leq 2^b$ and $\sqrt{m} \leq 2^{b/2}$. Therefore the running time is exponential in the input size b .

Exponentiation: recurrences and improved algorithms

To compute x^n naively, multiply x by itself n times:

$$x^n = \underbrace{x \cdot x \cdot \dots \cdot x}_{n \text{ multiplications}}.$$

If x has b bits, then x^n has $O(nb)$ bits. A coarse upper bound is that each multiplication costs $O((nb)^2)$, and there are n multiplications, giving $O(n^3b^2)$ bit operations.

Exponentiation by squaring uses a recurrence:

$$x^n = \begin{cases} (x^{n/2})^2, & n \text{ even,} \\ x \cdot (x^{(n-1)/2})^2, & n \text{ odd.} \end{cases}$$

This reduces the number of multiplications to $O(\log n)$. The precise bit complexity depends on the multiplication routine, but the algorithm is asymptotically much faster than the straight-line method.

Output-sensitive lower bounds: orthogonal segment intersections

Problem: given a set S of orthogonal line segments, count or report all intersection points. The naive algorithm checks every pair of segments and tests for intersection, which takes $O(n^2)$ time for n segments assuming a constant-time intersection test.

If the output size is h (the number of intersections), any reporting algorithm must take at least $\Omega(n + h)$ time, because it must read the input and write the output. Here $0 \leq h \leq \Theta(n^2)$.

4 Lecture 3

Computational Geometry: Orthogonal Segment Intersections

Problem Statement

Given a set S of n orthogonal line segments (either horizontal or vertical), report or count all intersection points. An intersection occurs where a horizontal segment crosses a vertical segment.

Naive Approach

Check all pairs of segments:

```

intersections = []
for each horizontal segment h:
    for each vertical segment v:
        if h and v intersect:
            intersections.append((h, v))

```

If we have n segments total, this is $O(n^2)$ intersection tests. Even if each test is $O(1)$, we have $O(n^2)$ total time.

Why Naive is Not Optimal

The key observation: the number of intersections h can be much smaller than n^2 . If the segments are sparse, h could be $O(n)$. An optimal algorithm should be output-sensitive, meaning its time depends on both n and h .

Lower bound: any algorithm must take at least $\Omega(n + h)$ time because:

- We must read all n segments (at least n time).
- We must write all h intersection points (at least h time).

The naive algorithm achieves $O(n^2)$ which can be much worse than $O(n + h)$ when h is small.

Sweep-Line Idea

Imagine a vertical line sweeping from left to right across the plane. At each point in time, this sweep line intersects some horizontal segments. The key insight:

A horizontal segment contributes its intersection to a vertical segment exactly when the sweep line reaches that vertical segment at the x -coordinate of intersection.

Event-Based Processing

Instead of checking all pairs, we process events in left-to-right order:

1. **Left endpoint of horizontal segment:** activate the segment (it now intersects the sweep line).
2. **Right endpoint of horizontal segment:** deactivate the segment.
3. **Vertical segment:** query which active horizontal segments intersect it.

At any x -coordinate, let S_x be the set of active (intersecting the sweep line) horizontal segments, sorted by y -coordinate.

Data Structure: Balanced Binary Search Tree

Maintain S_x in a balanced BST (AVL tree, Red-Black tree) keyed by y -coordinate. This supports:

- **Insert(y):** Add a horizontal segment at y -coordinate. Time: $O(\log n)$.
- **Delete(y):** Remove a horizontal segment. Time: $O(\log n)$.
- **Range Query(y_1, y_2):** Find all segments with y -coordinates in $[y_1, y_2]$. Time: $O(\log n + \text{output})$.

The range query works by finding the first segment with $y \geq y_1$ using binary search, then walking through successors until $y > y_2$.

Algorithm Pseudocode

```
sort all segment endpoints by x-coordinate
intersections = []
T = empty balanced BST # keyed by y-coordinate

for each event in left-to-right order:
    if event is left endpoint of horizontal segment h:
        T.insert(y-coordinate of h)
```

```

    else if event is right endpoint of horizontal segment h:
        T.delete(y-coordinate of h)

    else if event is vertical segment v:
        # v spans from y1 to y2
        for each h in T with y in [y1, y2]:
            intersections.append((h, v))

return intersections

```

Complexity Analysis

- Sorting: There are $2n$ segment endpoints (left and right of each horizontal segment, plus n vertical segments). Sorting takes $O(n \log n)$.
- Event processing:
 - Each insert/delete costs $O(\log n)$. There are $2n$ of them, so $O(n \log n)$ total.
 - Each vertical segment query costs $O(\log n + h_v)$ where h_v is the number of intersections for that vertical segment.
 - Summing over all vertical segments: $\sum_v O(\log n + h_v) = O(n \log n + h)$.

Total time: $O(n \log n) + O(n \log n + h) = O(n \log n + h)$

This matches the output-sensitive lower bound up to logarithmic factors! When h is small, this is much faster than $O(n^2)$.

Orthogonal segment intersections via sweep line (Original)

We revisit the problem of counting or reporting intersections between horizontal and vertical line segments. The naive $O(n^2)$ approach checks every pair; the goal is an output-sensitive algorithm.

Sweep-line idea (Original)

Imagine a vertical sweep line moving from left to right. At any x -coordinate, define S_x as the set of horizontal segments that intersect the sweep line, sorted by their y -coordinates.

Observation: S_x changes only at endpoints of horizontal segments. Between consecutive event points, the sorted set is constant. With n horizontal segments there are at most $2n$ distinct sets $S_1, S_2, \dots, S_{2n}$.

Event queue

Events are:

1. Left endpoint of a horizontal segment: insert it into S_x .
 2. Right endpoint of a horizontal segment: delete it from S_x .
 3. A vertical segment at $x = x_v$: query S_x for all intersections.
- The event queue is the list of all such events sorted by x -coordinate.

Data structure for S_x

Maintain S_x in a balanced BST keyed by y (AVL, red-black, etc.). This supports:

- Insert/Delete a horizontal segment: $O(\log n)$.
- Search for the first active segment with $y \geq y_1$: $O(\log n)$.
- Successor iteration to report all segments with $y \leq y_2$.

Reporting intersections for a vertical segment

For a vertical segment spanning $y \in [y_1, y_2]$ at $x = x_v$:

1. Find the first active horizontal segment with $y \geq y_1$.
2. Walk successors until $y > y_2$, reporting each intersection.

If $h(v)$ intersections are reported for this vertical segment, the work is $O(\log n + h(v))$.

Running time

There are at most $2n$ horizontal endpoints, so sorting the event queue takes $O(n \log n)$. During the sweep:

- Endpoint updates take $O(\log n)$ each, for $O(n \log n)$ total.
- Vertical segment queries cost $\sum_v O(\log n + h(v))$.

Overall:

$$O(n \log n) + \sum_v O(\log n + h(v)) = O(n \log n + h),$$

where h is the total number of intersections. This matches the output-sensitive lower bound up to logarithmic factors.

Note on non-orthogonal segments

The sweep-line framework extends to general segment intersections, but additional event types and geometric ordering are needed (and the algorithm becomes more complex).

5 Lecture 4: Sorting and Radix Algorithms

Comparison-Based Sorting: Lower Bound

Before discussing specific algorithms, it's important to understand the fundamental limits.

Decision Tree Model

Any comparison-based sorting algorithm can be viewed as a decision tree:

- Each internal node represents a comparison (e.g., "is $a_i < a_j$?").
- Each leaf represents a final sorted order (permutation).

For n distinct elements, there are $n!$ possible sorted orders. A binary decision tree with L leaves has height at least $\lceil \log_2 L \rceil$. Therefore:

$$\text{Minimum comparisons} \geq \log_2(n!) = \Theta(n \log n)$$

Using Stirling's approximation: $\log_2(n!) \approx n \log_2 n - 1.44n$.

Conclusion: No comparison-based sorting algorithm can beat $\Omega(n \log n)$ in the worst case. Mergesort and heapsort are asymptotically optimal for comparison sorting.

Non-Comparison Sorting

For special cases where keys are restricted to a bounded range, we can sort faster than $O(n \log n)$ by avoiding comparisons.

Counting Sort: Sorting a Small Range

Problem

Sort n integers where each integer lies in the range $\{0, 1, \dots, m - 1\}$.

Algorithm

```
def counting_sort(A, m):
    # A is array of length n, values in [0, m-1]
    count = [0] * m # count[i] = number of elements equal to i

    # Count occurrences
    for x in A:
        count[x] += 1

    # Cumulative sum (count[i] = position of first element >= i)
    cumsum = 0
    for i in range(m):
        temp = count[i]
        count[i] = cumsum
        cumsum += temp

    # Place elements in correct positions
    result = [0] * len(A)
    for x in A:
        result[count[x]] = x
        count[x] += 1

    return result
```

Example

Input: $A = [2, 0, 2, 1, 1, 0]$, $m = 3$

Step 1: Count occurrences

- $\text{count} = [2, 2, 2]$ (two 0's, two 1's, two 2's)

Step 2: Output sorted array by writing each element i exactly $\text{count}[i]$ times

- Output: $[0, 0, 1, 1, 2, 2]$

Complexity

- Counting: $O(n)$ to scan input.
- Iterating buckets: $O(m)$ to output.
- Total: $O(n + m)$

When $m = O(n)$, this is linear time—faster than comparison sorting!

Stability

An algorithm is stable if equal elements maintain their relative order from the input. Counting sort can be made stable by processing elements in reverse order when placing them. Stability is crucial for radix sort.

Radix Sort: Sorting Large Ranges

Problem

Sort n integers in the range $\{0, 1, \dots, B^k - 1\}$ where B is a base (like 10 or 2^{16}).

Key Insight

View each number as a k -digit number in base B :

$$\text{number} = d_{k-1}B^{k-1} + d_{k-2}B^{k-2} + \dots + d_1B + d_0$$

where each $d_i \in \{0, 1, \dots, B - 1\}$.

Algorithm

```
def radix_sort(A, k, B):  
    # A is array of n numbers, each a k-digit number in base B  
  
    for digit_position in range(k):  
        # Stable sort by digit at this position  
        A = stable_sort_by_digit(A, digit_position, B)  
  
    return A
```

where 'stable_sort_by_digit' uses counting sort to sort by a single digit, using $O(n + B)$ time.

Example: Sorting in decimal ($B=10$, $k=3$)

Input: [329, 457, 657, 839, 436, 720, 355]

- **Pass 1** (ones place):
[720, 329, 457, 436, 355, 657, 839]
- **Pass 2** (tens place):
[329, 436, 355, 457, 657, 720, 839]
- **Pass 3** (hundreds place):
[329, 355, 436, 457, 657, 720, 839]

Why this works: After sorting by digit i , if two numbers differ in digit $j > i$, the one with the larger digit is correctly positioned. For numbers with the same digit at position i , the stable sort preserves their relative order from the previous pass.

Complexity

Each of the k passes takes $O(n + B)$ using counting sort. Total time:

$$O(k(n + B))$$

Choosing $B = \Theta(n)$ (e.g., $B = n$):

$$O(k \cdot 2n) = O(kn)$$

For numbers in $\{0, 1, \dots, n^k - 1\}$, we have $k \log_B n^k = k$ digits in base B , so radix sort takes $O(kn)$ linear time.

When to Use Radix Sort

- Small values of k relative to n .
- Integer keys (not strings or floating-point).
- Practical for sorting large datasets of integers.

Lexicographic Ordering of Strings

Problem

Sort n strings lexicographically (dictionary order).

Solution Using Radix Sort

Treat strings as base- Σ numbers where Σ is the alphabet size:

1. Pad all strings to equal length L using a special "end-of-string" symbol $\$$ (smaller than any character).
2. Apply radix sort from right to left (least significant to most significant).

Example

Strings: ["cat", "car", "dog", "can"]

Padded (length 3): "cat", "car", "dog", "can"

Radix sort from right to left:

- Right column: a, a, g, a $\rightarrow$ sorted order by third character.
- Middle column: t, r, o, n $\rightarrow$ sorted order by second character (stable, preserves third).
- Left column: c, c, d, c $\rightarrow$ sorted order by first character (stable).

Result: ["can", "car", "cat", "dog"]

Complexity for Strings

If there are n strings with total length $N = \sum_i \ell_i$ (where ℓ_i is the length of string i), and maximum length L :

$$O(L(n + \Sigma))$$

where Σ is the alphabet size. This respects the true input size N , not nL , because we radix sort characters (each character is an input bit).

Stability

A sorting algorithm is *stable* if equal keys appear in the output in the same order as in the input. Stability is essential for radix sort because earlier digit-ordering must be preserved.

Radix sort

Radix sort performs a stable sort on each digit, from least significant to most significant. If numbers have k digits in base B , the running time is $O(k(n + B))$ using counting sort per digit. Choosing $B = \Theta(n)$ gives $O(kn)$ time for values in $[0, n^k - 1]$.

Hexadecimal digits are 0–15 (written as 0, 1, ..., 9, A, ..., F), which is a common base for digit-based sorting examples.

Lexicographic order of strings

Lexicographic order can be achieved by radix sorting characters from right to left, provided all strings have the same length. Introduce a special end symbol \$ smaller than any character, and pad shorter strings with \$ so equal-length comparison matches lexicographic order.

If the string lengths are $\ell_1, \dots, \ell_n$ with $\sum_i \ell_i = N$ and $\ell_i \leq L$, the true input size is N (not nL). Radix sorting characters respects this input-size view.

Graphs: basic definitions

A graph $G = (V, E)$ has vertex set V and edge set E . Graphs can be:

- **Directed:** edges are ordered pairs (u, v) .
- **Undirected:** edges are unordered.

For a weighted graph, a weight function maps edges to real numbers, $w : E \rightarrow \mathbb{R}$.

Connectivity and traversal

An undirected graph is connected if every pair of vertices is connected by a path. Use DFS or BFS to test connectivity. With an adjacency list representation, both DFS and BFS run in $O(|V| + |E|) = O(n + m)$ time.

Spanning trees and minimum spanning trees

A *spanning tree* is a connected subgraph that spans all vertices with the minimum possible number of edges (a tree). In a weighted graph, the *minimum spanning tree* (MST) is a spanning tree with minimum total weight.

6 Lecture 5: Maximal Points and Geometric Algorithms

Maximal Points Problem

Definition

Given a set of 2D points $S = \{(x_1, y_1), \dots, (x_n, y_n)\}$, a point p is **maximal** (or **Pareto optimal**) if no other point has both coordinates greater than or equal to p , with at least one strictly greater.

Formally, $p = (x_p, y_p)$ is maximal if:

$$\forall q \in S \setminus \{p\} : \quad x_q \leq x_p \text{ or } y_q \leq y_p$$

Equivalently: p is not dominated by any other point.

Examples

Consider points: $(1, 9), (2, 7), (3, 5), (4, 4), (5, 3)$

- $(1, 9)$ is maximal (highest y).
- $(2, 7)$ is not maximal (dominated by $(1, 9)$).
- $(3, 5)$ is not maximal (dominated by $(1, 9)$).
- $(4, 4)$ is not maximal (dominated by $(1, 9)$).
- $(5, 3)$ is maximal (furthest right).

Maximal points: $\{(1, 9), (5, 3)\}$.

Properties of Maximal Points

1. When sorted by x -coordinate in increasing order, the y -coordinates are in decreasing order (forms a "staircase").
2. There are at most n maximal points, often much fewer.
3. The set of maximal points is unique.

Naive Algorithm

```
maximal = []
for each point p in S:
    is_maximal = true
    for each point q in S:
        if q != p and q.x > p.x and q.y > p.y:
            is_maximal = false
            break
    if is_maximal:
        maximal.append(p)
return maximal
```

Complexity: $O(n^2)$ — we check each of n points against all n others.

Divide-and-Conquer Approach

Intuition

1. Divide points into left and right halves (by median x -coordinate).
2. Recursively find maximal points in each half.
3. Merge the two sets.

Key Insight for Merging

All points in the right half have x -coordinates at least as large as those in the left. Therefore:

Every maximal point in the right half remains maximal in the union.

A maximal point from the left survives merging only if its y -coordinate exceeds all maximal points on the right.

Algorithm

```
def find_maximal(points):
    if len(points) == 1:
        return points

    # Divide by median x-coordinate
    sorted_by_x = sort(points, key=lambda p: p.x)
    mid = len(sorted_by_x) // 2
    left = sorted_by_x[:mid]
    right = sorted_by_x[mid:]

    # Conquer: recursively find maximal points
    left_maximal = find_maximal(left)
    right_maximal = find_maximal(right)
```

```

# Merge: right maximal points are all maximal in the union
# For left maximal points, keep only those with y > max(y in right)
max_y_right = max(p.y for p in right_maximal)

merged = right_maximal.copy()
for p in left_maximal:
    if p.y > max_y_right:
        merged.append(p)

# Sort by decreasing x for next recursion (maintains staircase)
return sort(merged, key=lambda p: -p.x)

```

Merge Complexity

The merge step takes $O(n)$ time:

- Computing $\max y$ in right: $O(|right|)$
- Filtering left points: $O(|left|)$
- Total: $O(|left| + |right|) = O(n)$

Even better: since maximal points form a staircase, they're already sorted. We can merge in $O(|left| + |right|)$ time by walking pointers.

Overall Complexity

$$T(n) = 2T(n/2) + O(n) = \Theta(n \log n)$$

by the Master Theorem. This is optimal (requires sorting by x -coordinate anyway).

Maximal points and dominance (Original text continues below)

Given a set of points $S = \{(x_i, y_i)\}_{i=1}^n$, a point $p \in S$ is *maximal* if no other point is simultaneously to its upper-right. Formally:

$$\forall q \in S \setminus \{p\}, \quad x(q) < x(p) \text{ or } y(q) < y(p).$$

Define *dominance*: point p *dominates* q if

$$x(p) > x(q) \text{ and } y(p) > y(q).$$

Then p is maximal iff no point in S dominates it. The two definitions are equivalent because "not dominated" is exactly the negation of the existence of a point with both coordinates larger.

Maximal points form a *staircase* (also called the Pareto frontier): when sorted by increasing x , their y -coordinates strictly decrease.

Goal

Design an $O(n \log n)$ algorithm to find all maximal points.

Naive algorithm

For each point, scan all others and check for a dominating point. This takes $O(n^2)$ time.

Divide and conquer approach

1. Divide S into two sets of size $n/2$, typically by the median x -coordinate.
2. Recursively compute maximal points in the left and right sets.
3. Combine the two maximal sets in $O(n)$ time.

Linear-time combine

Because all points in the right set have x -coordinates at least as large as those in the left, every maximal point in the right set remains maximal in the union. A maximal point from the left survives only if its y -coordinate is larger than every maximal point on the right.

If the maximal points in each half are stored in decreasing x order (the staircase order), we can merge by sweeping from right to left, maintaining the maximum y seen in the right set and discarding left points with $y \leq y_{\max}$. This takes linear time in the number of maximal points.

Running time

The recurrence is

$$T(n) = 2T(n/2) + O(n),$$

which solves to $T(n) = O(n \log n)$.

7 Lecture 6: Divide-and-Conquer and Polynomial Multiplication

Divide-and-Conquer Paradigm

Template

1. **Divide:** Split the input into k independent subproblems (usually $k = 2$).
2. **Conquer:** Recursively solve each subproblem.
3. **Combine:** Merge the solutions into the final answer.

General Recurrence

$$T(n) = \sum_{i=1}^k T(n_i) + P(n) + M(n)$$

where:

- $P(n)$ = partitioning cost
- $M(n)$ = merging/combination cost
- $\sum_i n_i = n$

Classic Example: Mergesort

Algorithm

```
def mergesort(A):
    if len(A) <= 1:
        return A

    mid = len(A) // 2
    left = mergesort(A[:mid])
```

```

    right = mergesort(A[mid:])

    return merge(left, right)

def merge(L, R):
    result = []
    i = j = 0
    while i < len(L) and j < len(R):
        if L[i] <= R[j]:
            result.append(L[i])
            i += 1
        else:
            result.append(R[j])
            j += 1
    result.extend(L[i:])
    result.extend(R[j:])
    return result

```

Analysis

- Dividing: $O(1)$
 - Recursion: $2T(n/2)$
 - Merging: $O(n)$
- Recurrence: $T(n) = 2T(n/2) + O(n)$
 By the Master Theorem: $T(n) = \Theta(n \log n)$

Fibonacci: Exponential Recurrence

Definition

$$F(n) = F(n-1) + F(n-2), \quad F(0) = 0, \quad F(1) = 1$$

Naive Recursive Implementation

```

def fib_naive(n):
    if n <= 1:
        return n
    return fib_naive(n-1) + fib_naive(n-2)

```

Problem

The recursion tree has exponential depth. The number of function calls is $\Theta(F(n))$, which is exponential!

$$F(n) = \Theta(\phi^n), \quad \text{where } \phi = \frac{1 + \sqrt{5}}{2} \approx 1.618 \text{ (golden ratio)}$$

For $n = 40$, this is $\sim 10^8$ calls. For $n = 100$, it's infeasible.

Fast Fibonacci via Matrix Exponentiation

Key insight: Express Fibonacci as matrix multiplication:

$$\begin{bmatrix} F_n \\ F_{n-1} \end{bmatrix} = \begin{bmatrix} 1 & 1 \\ 1 & 0 \end{bmatrix} \begin{bmatrix} F_{n-1} \\ F_{n-2} \end{bmatrix}$$

Therefore:

$$\begin{bmatrix} F_n \\ F_{n-1} \end{bmatrix} = \begin{bmatrix} 1 & 1 \\ 1 & 0 \end{bmatrix}^{n-1} \begin{bmatrix} 1 \\ 0 \end{bmatrix}$$

Using fast exponentiation (exponentiation by squaring), we compute M^{n-1} in $O(\log n)$ matrix multiplications. Each 2×2 matrix multiplication takes $O(1)$ time, so:

$$\text{Time} = O(\log n)$$

```
def matrix_mult(A, B):
    return [[A[0][0]*B[0][0] + A[0][1]*B[1][0],
            A[0][0]*B[0][1] + A[0][1]*B[1][1],
            A[1][0]*B[0][0] + A[1][1]*B[1][0],
            A[1][0]*B[0][1] + A[1][1]*B[1][1]]

def matrix_pow(M, n):
    if n == 1:
        return M
    if n % 2 == 0:
        half = matrix_pow(M, n // 2)
        return matrix_mult(half, half)
    else:
        return matrix_mult(M, matrix_pow(M, n - 1))

def fib_fast(n):
    M = [[1, 1], [1, 0]]
    result = matrix_pow(M, n)
    return result[0][1] # Returns F_n
```

Comparison

- Naive: $O(\phi^n)$ — exponential, infeasible for $n > 40$.
- Fast: $O(\log n)$ — logarithmic, feasible for $n = 10^{18}$.

Integer Multiplication: From Quadratic to Subquadratic

Problem

Multiply two n -bit integers A and B .

Grade-School Algorithm

```
def multiply_naive(A, B, n):
    result = 0
    for i in range(n):
        result += (get_bit(A, i) * B) << i
    return result
```

This performs n shifts and n additions of n -bit numbers, each taking $O(n)$ time. Total: $O(n^2)$.

Karatsuba Algorithm

Key Idea: Reduce the number of multiplications.

Split each n -bit number into high and low halves:

$$A = A_1 \cdot 2^{n/2} + A_0, \quad B = B_1 \cdot 2^{n/2} + B_0$$

Naive multiplication requires 4 sub-multiplications:

$$AB = A_1B_1 \cdot 2^n + (A_1B_0 + A_0B_1) \cdot 2^{n/2} + A_0B_0$$

Cost: $4 \times M(n/2)$ where $M(n/2)$ is the cost of multiplying $(n/2)$ -bit numbers.

Karatsuba's Trick: Compute only 3 multiplications:

$$P = (A_1 + A_0)(B_1 + B_0)$$

Then:

$$A_1B_0 + A_0B_1 = P - A_1B_1 - A_0B_0$$

So:

$$AB = A_1B_1 \cdot 2^n + (P - A_1B_1 - A_0B_0) \cdot 2^{n/2} + A_0B_0$$

Recurrence: $M(n) = 3M(n/2) + O(n)$

By Master Theorem: $M(n) = O(n^{\log_2 3}) = O(n^{1.585})$

Improvement: From $O(n^2)$ to $O(n^{1.585})$ — substantial for large n .

Toom-Cook and Further Improvements

Generalizing to k -way splits reduces the exponent further. With large k and FFT-based polynomial multiplication, we can achieve $O(n \log n)$ integer multiplication using Schönhage-Strassen algorithm.

The running time can be written as

$$T(n) = \sum_{i=1}^k T(n_i) + P(n) + M(n), \quad \text{with } \sum_i n_i = n,$$

where $P(n)$ is the partition cost and $M(n)$ is the merge/combination cost.

Classic example: mergesort

Mergesort splits into two halves and merges in linear time:

$$T(n) = 2T(n/2) + O(n) \Rightarrow T(n) = O(n \log n).$$

Recurrences that do not fit the Master Theorem

Some divide-and-conquer recurrences are not of the form $T(n) = aT(n/b) + f(n)$, e.g.

$$T(n) = \sqrt{n} T(\sqrt{n}) + O(n^{3/2} \log^2 n),$$

which motivates more general techniques (substitution, recursion trees, etc.).

Fibonacci recurrence and growth

The Fibonacci sequence is

$$F(n) = F(n-1) + F(n-2), \quad F(0) = 0, \quad F(1) = 1.$$

The naive recursive algorithm makes a number of calls proportional to $F(n)$, so its running time is exponential. The closed-form growth is

$$F(n) = \Theta(\varphi^n), \quad \varphi = \frac{1 + \sqrt{5}}{2},$$

and $\log F(n) \approx n \log \varphi$.

Matrix formulation of Fibonacci

Define

$$\begin{bmatrix} F_n \\ F_{n-1} \end{bmatrix} = \begin{bmatrix} 1 & 1 \\ 1 & 0 \end{bmatrix} \begin{bmatrix} F_{n-1} \\ F_{n-2} \end{bmatrix}.$$

Then

$$\begin{bmatrix} F_n \\ F_{n-1} \end{bmatrix} = \begin{bmatrix} 1 & 1 \\ 1 & 0 \end{bmatrix}^{n-1} \begin{bmatrix} F_1 \\ F_0 \end{bmatrix}.$$

Using fast exponentiation, this yields $O(\log n)$ matrix multiplications, giving a much faster Fibonacci algorithm.

Multiplying two n -bit integers

Let

$$A = A_1 2^{n/2} + A_0, \quad B = B_1 2^{n/2} + B_0,$$

where A_1, A_0, B_1, B_0 are $n/2$ -bit numbers. Then

$$AB = A_1 B_1 2^n + (A_1 B_0 + A_0 B_1) 2^{n/2} + A_0 B_0.$$

The straightforward divide-and-conquer uses 4 multiplications of size $n/2$:

$$M(n) = 4M(n/2) + O(n) \Rightarrow M(n) = O(n^2).$$

Karatsuba's idea (3 multiplications)

Compute:

$$P = (A_1 + A_0)(B_1 + B_0).$$

Then

$$A_1 B_0 + A_0 B_1 = P - A_1 B_1 - A_0 B_0.$$

So we need only three multiplications of size $n/2$:

$$T(n) = 3T(n/2) + O(n).$$

This solves to

$$T(n) = O(n^{\log_2 3}) \approx O(n^{1.585}).$$

General k -way split (Toom-Cook viewpoint)

Split into k blocks of size n/k and write

$$A(x) = A_{k-1}x^{k-1} + \cdots + A_1x + A_0, \quad B(x) = B_{k-1}x^{k-1} + \cdots + B_1x + B_0,$$

with $x = 2^{n/k}$. Then AB equals $C(x)$ where $C(x) = A(x)B(x)$ is a degree $2k - 2$ polynomial.

To obtain C , evaluate A and B at $2k - 1$ distinct points, multiply pointwise, and interpolate. This yields the recurrence (for constant k):

$$T(n) = (2k - 1)T(n/k) + O(n).$$

With larger constant k , the exponent approaches 1, giving $O(n^{1+\varepsilon})$ time for any fixed $\varepsilon > 0$.

Polynomial interpolation reminder

A degree- d polynomial is uniquely determined by its values at $d + 1$ distinct points. For example, for degree 3:

$$\begin{bmatrix} x_1^3 & x_1^2 & x_1 & 1 \\ x_2^3 & x_2^2 & x_2 & 1 \\ x_3^3 & x_3^2 & x_3 & 1 \\ x_4^3 & x_4^2 & x_4 & 1 \end{bmatrix} \begin{bmatrix} a_3 \\ a_2 \\ a_1 \\ a_0 \end{bmatrix} = \begin{bmatrix} y_1 \\ y_2 \\ y_3 \\ y_4 \end{bmatrix}.$$

Toward $O(n \log n)$ multiplication

When k grows with n (e.g., $k \approx \sqrt{n}$) and evaluation/interpolation use roots of unity, FFT gives $O(n \log n)$ polynomial multiplication, leading to near-linear-time integer multiplication.

8 Lecture 7: Order Statistics and Selection

Selection Problem

Definition

Given a set $S = \{x_1, x_2, \dots, x_n\}$ and an integer k with $1 \leq k \leq n$, find the k -th smallest element (the element of **rank** k).

Examples

$S = [3, 1, 4, 1, 5, 9, 2, 6]$ (after removing duplicates: $[1, 2, 3, 4, 5, 6, 9]$)

- $k = 1$: minimum = 1
- $k = 4$: median = 4
- $k = 7$: maximum = 9

Why is Selection Important?

- Computing medians (for statistics, algorithms like quicksort, data analysis).
- Finding k -nearest neighbors.
- Percentile calculations.

Simple Approaches

Sort and Pick

```
def select_by_sort(S, k):
    sorted_S = sort(S)
    return sorted_S[k-1]  # k-1 because 0-indexed
```

Time: $O(n \log n)$. This works but is overkill since we don't need the full sorted order.

Min-Heap Extraction

```
def select_by_heap(S, k):
    heap = build_min_heap(S)  #  $O(n)$ 
    for i in range(k-1):
        extract_min(heap)      #  $O(\log n)$  each
    return extract_min(heap)
```

Time: $O(n + k \log n)$. Good when k is small, but bad when k is large (close to n).

Partition-Based Selection (Linear Time on Average)

Quickselect: Randomized Version

The key insight: use partitioning (like in quicksort) to narrow down the search.

```
def quickselect(S, k):
    if len(S) == 1:
        return S[0]

    # Pick random pivot
    pivot_idx = random.randint(0, len(S)-1)
    pivot = S[pivot_idx]

    # Partition: L < pivot, M = pivot, R > pivot
    L = [x for x in S if x < pivot]
    M = [x for x in S if x == pivot]
    R = [x for x in S if x > pivot]

    # Recurse on correct part
    if k <= len(L):
        return quickselect(L, k)
    elif k <= len(L) + len(M):
        return pivot
    else:
        return quickselect(R, k - len(L) - len(M))
```

Analysis

Define a "good" pivot as one with rank r where $n/4 \leq r \leq 3n/4$.

Claim: A random pivot is good with probability $\geq 1/2$.

Proof: The good ranks are in the range $[n/4, 3n/4]$, which contains $n/2$ elements out of n total. By symmetry, picking a random element gives probability $\geq 1/2$ of being good.

Recurrence: If we always pick a good split, we recurse on at most $3n/4$ elements:

$$T(n) \leq T(3n/4) + O(n)$$

This solves to $T(n) = O(n)$ (by the recursion tree method or Master Theorem).

With Randomization: Let $N(n)$ be the expected number of elements examined. In iteration i , we have $E[\text{size}_i]$:

- With probability $1/2$, we pick a good pivot and get $\leq 3n/4$ elements.
- With probability $1/2$, we pick a bad pivot and might get $\leq n - 1$ elements, but on average less.

The expected recurrence is:

$$E[T(n)] \leq E[T(3n/4)] + O(n) = O(n)$$

Example

$S = [3, 1, 4, 1, 5, 9, 2, 6]$, find $k = 4$:

- Pick pivot = 5 (rank 7). Too large, recurse on $L = [3, 1, 4, 1, 2, 6]$ (the part < 5). But wait, this example shows we might not get a perfectly balanced split.
- Better example: pivot = 4 (rank 4). Exactly right! Return 4.

Deterministic Linear Time: Median of Medians

Why Deterministic?

Quickselect works well in expectation, but in the worst case (unlucky random choices), it still takes $O(n^2)$ time. For guaranteed linear time, we need a deterministic algorithm.

Key Idea: Guaranteed Good Pivot

Find a pivot whose rank is guaranteed to be in a constant-fraction range, without relying on randomness.

Algorithm: Median-of-Medians

```
def median_of_medians(S):
    # Base case
    if len(S) <= 5:
        return sorted(S)[len(S) // 2]

    # Step 1: Partition S into groups of 5
    groups = [S[i:i+5] for i in range(0, len(S), 5)]

    # Step 2: Find median of each group
    medians = [sorted(group)[len(group)//2] for group in groups]

    # Step 3: Recursively find median of medians
    mom = median_of_medians(medians)

    return mom

def select_with_median_of_medians(S, k):
```

```

if len(S) == 1:
    return S[0]

# Find pivot using median-of-medians
pivot = median_of_medians(S)

# Partition around pivot
L = [x for x in S if x < pivot]
M = [x for x in S if x == pivot]
R = [x for x in S if x > pivot]

# Recurse
if k <= len(L):
    return select_with_median_of_medians(L, k)
elif k <= len(L) + len(M):
    return pivot
else:
    return select_with_median_of_medians(R, k - len(L) - len(M))

```

Why the Guarantee Works

The median-of-medians has a special property: in a split of n elements:

- At least $3n/10$ elements are $\geq$ median-of-medians.
- At least $3n/10$ elements are $\leq$ median-of-medians.

Proof sketch:

1. We have $\lceil n/5 \rceil$ groups.
2. The median-of-medians is the median of $\lceil n/5 \rceil$ medians.
3. Half of the $\lceil n/5 \rceil$ medians are $\geq$ the median-of-medians (roughly $n/10$ medians).
4. Each such median has 2 elements $\geq$ it in its group, giving $\geq 2 \cdot n/10 = n/5$ elements.
5. Additional elements: $\geq n/5 + n/10 = 3n/10$.

Thus the recursion is:

$$T(n) = T(\lceil n/5 \rceil) + T(7n/10) + O(n)$$

Solving the Recurrence

By induction, we can show $T(n) \leq cn$ for some constant c :

Assume $T(m) \leq cm$ for all $m < n$. Then:

$$T(n) \leq c \cdot \frac{n}{5} + c \cdot \frac{7n}{10} + O(n) = cn \left(\frac{1}{5} + \frac{7}{10} \right) + O(n) = cn \cdot \frac{9}{10} + O(n)$$

Since $9/10 < 1$, this is $\leq cn$ for sufficiently large c . Thus $T(n) = \Theta(n)$.

Practical vs. Theoretical

- **Theoretical:** Median-of-medians is optimal ($O(n)$ guaranteed).
- **Practical:** Quickselect with good random pivots is often faster in practice due to better constant factors.
- **Hybrid:** Use quickselect, but fall back to median-of-medians if too many iterations without a good pivot.

Baseline methods

- Sort and return the k -th element: $O(n \log n)$.
- Repeatedly extract the minimum k times: $O(kn)$ (bad if k is large).

The goal is a linear-time selection algorithm without fully sorting.

Partition-based selection

Partition around a pivot p (as in quicksort) and let r be the rank of p :

- If $k = r$, return p .
- If $k < r$, recurse on the left partition.
- If $k > r$, recurse on the right partition with $k \leftarrow k - r$.

Each partition step costs $O(n)$.

Randomized selection (expected linear time)

If the pivot is chosen uniformly at random, it is a *good splitter* (rank between $n/4$ and $3n/4$) with probability $1/2$. Conditioning on a good split gives the recurrence

$$T(n) \leq n + T(3n/4).$$

The number of trials before a good split is geometric with mean 2, so

$$\mathbb{E}[T(n)] = O(n).$$

Using Markov's inequality, $\Pr[T > \alpha n] \leq 1/\alpha$, giving a tail bound on runtime.

Deterministic linear time: median of medians

Key idea: choose a pivot whose rank is guaranteed to be in a constant middle range.

Algorithm outline:

1. Partition S into groups of 5 and sort each group.
2. Let m_i be the median of each group; recursively find the median M of the m_i 's.
3. Use M as the pivot and partition S into $S_{<}, M, S_{>}$.
4. Recurse into the side containing the k -th element.

At least $3n/10$ elements are $\leq M$ and at least $3n/10$ are $\geq M$, so the larger recursive subproblem has size at most $7n/10$. The recurrence is

$$T(n) = T(n/5) + T(7n/10) + O(n) = O(n).$$

Approximate median

An *approximate median* is an element whose rank lies in $[\alpha n, (1 - \alpha)n]$ for some constant $0 < \alpha < 1$. The median-of-medians algorithm guarantees such a pivot and thus linear time.

9 Lecture 8: Comparison-Based Sorting Lower Bounds

Comparison-Based Sorting Lower Bound (Detailed)

Decision Tree Model

Any comparison-based sorting algorithm can be modeled as a decision tree:

- Each internal node represents a binary comparison (e.g., "is $a_i < a_j$?").
- Each leaf represents a sorted permutation of the input.
- Path from root to leaf represents the sequence of comparisons made for a given input.

Fundamental Result

For n distinct elements, there are $n!$ possible orderings. Since a decision tree must distinguish between all $n!$ orderings, it must have at least $n!$ leaves.

A binary tree with L leaves has height $\geq \lceil \log_2 L \rceil$. Therefore:

$$\text{Worst-case comparisons} \geq \log_2(n!) = \Theta(n \log n)$$

Using Stirling's approximation: $\log_2(n!) \approx n \log_2 n - 1.44n$.

Corollary: No comparison-based sorting algorithm can achieve better than $\Theta(n \log n)$ worst-case time.

Why Mergesort and Heapsort are Optimal

Both algorithms sort n elements using $O(n \log n)$ comparisons in the worst case, matching the lower bound.

Average-Case Lower Bound

The average-case analysis also gives $\Omega(n \log n)$:

$$\text{Average comparisons} = \sum_{\text{leaves}} \text{depth}(\text{leaf}) \cdot \text{prob}(\text{leaf}) \geq \log_2(n!)$$

This holds for any probability distribution of inputs.

10 Lecture 9 10: Heaps and Priority Queues

Binary Heaps: Overview

A binary heap is a complete binary tree that satisfies the heap property. For a **min-heap**:

$$\text{key}(\text{parent}) \leq \text{key}(\text{child})$$

Array Representation

Store the heap in an array where:

- Node at index i has left child at $2i + 1$, right child at $2i + 2$, parent at $\lfloor (i - 1)/2 \rfloor$.
- The tree is a complete binary tree (all levels full except possibly the last).

Operations

```
def build_min_heap(A):
    n = len(A)
    for i in range(n // 2, -1, -1):
        min_heapify(A, i)
```

```

def min_heapify(A, i):
    # Enforce min-heap property at node i
    n = len(A)
    smallest = i
    left = 2 * i + 1
    right = 2 * i + 2

    if left < n and A[left] < A[smallest]:
        smallest = left
    if right < n and A[right] < A[smallest]:
        smallest = right

    if smallest != i:
        A[i], A[smallest] = A[smallest], A[i]
        min_heapify(A, smallest)

```

Analysis: $O(n)$ time. Counter-intuitive: although we call ‘min_heapify’ for $n/2$ nodes, most calls are on leaf nodes.

```

def extract_min(heap):
    if len(heap) == 0:
        raise Error("Heap is empty")
    min_val = heap[0]
    heap[0] = heap[-1]
    heap.pop()
    min_heapify(heap, 0)
    return min_val

```

Analysis: $O(\log n)$ time due to the heapify operation.

```

def insert(heap, key):
    heap.append(key)
    i = len(heap) - 1
    # Bubble up
    while i > 0 and heap[(i-1)//2] > heap[i]:
        heap[i], heap[(i-1)//2] = heap[(i-1)//2], heap[i]
        i = (i - 1) // 2

```

Analysis: $O(\log n)$ time; path from leaf to root has length $\leq \log n$.

Heapsort

```

def heapsort(A):
    build_min_heap(A)
    result = []
    while A:
        result.append(extract_min(A))
    return result

```

Complexity: $O(n \log n)$ — optimal for comparison-based sorting.

Binomial Heaps

Binomial Trees

A **binomial tree** B_i is defined recursively:

- B_0 is a single node.
- B_i is formed by linking two B_{i-1} trees (making one root a child of the other).

Properties of B_i :

- Number of nodes: 2^i
- Height: i
- Root degree: i
- Number of nodes at depth j : $\binom{i}{j}$

Binomial Heap Structure

A binomial heap is a collection of binomial trees satisfying the heap property. For a heap with n elements, we have at most $\lfloor \log_2 n \rfloor + 1$ trees (one for each bit in the binary representation of n).

Key Insight: If $n = 13 = 1101_2$, we have trees B_3, B_2, B_0 (representing $8 + 4 + 1$).

Operations

Merge Merge two binomial heaps by linking trees of the same order (like binary addition with carries):

```
def merge_binomial_heaps(H1, H2):
    trees = {}

    # Collect all trees from both heaps
    for tree in H1 + H2:
        order = tree.order
        while order in trees:
            # Link two trees of the same order
            tree = link(tree, trees[order])
            order += 1
        trees[order] = tree

    return BinomialHeap(sorted(trees.values(), key=lambda t: t.order))
```

Time: $O(\log n)$ since we process at most $\log n$ orders.

Extract Min

1. Find the tree with minimum root.
 2. Remove that tree's root.
 3. Merge the children (which form a binomial heap) with the remaining trees.
- Time: $O(\log n)$.

Insert Create a single-node tree and merge it with the existing heap.

Time: $O(\log n)$ worst case, $O(1)$ amortized (analyzed later).

11 Lecture 11 12: Amortized Analysis

Amortized Analysis: Accounting Method

Instead of analyzing each operation individually, we account for groups of operations.

Example: Binary Counter

Consider a counter with n bits. Each increment flips some trailing 1s to 0s and flips one 0 to 1.

Naive worst-case: If all n bits are 1, incrementing flips all n bits, costing $O(n)$.

Amortized analysis: Over 2^n increments from 0 to $2^n - 1$:

- Bit 0 flips 2^n times (every increment).
- Bit 1 flips 2^{n-1} times (every 2 increments).
- Bit i flips 2^{n-i} times.

Total flips: $\sum_{i=0}^{n-1} 2^{n-i} = 2^n + 2^{n-1} + \dots + 2 = 2^{n+1} - 2 < 2^{n+1}$.

Amortized cost per increment: $(2^{n+1})/2^n = 2 = O(1)$.

Potential Method

Assign a potential function Φ to each data structure state. The amortized cost is:

$$\hat{c}_i = c_i + (\Phi_i - \Phi_{i-1})$$

where c_i is the actual cost of operation i .

Example: Binomial Heap Insertion

Let Φ = number of trees in the heap. Inserting a new node:

- Creates a new single-node tree: cost $O(1)$, $\Delta\Phi = +1$.
- Merging links trees of the same order, reducing the number of trees.

In the best case, the new tree immediately merges with an existing tree, reducing the count. The amortized cost remains $O(1)$.

12 Lecture 13: Minimum Spanning Trees

MST Problem

Definition

Given a connected, undirected, weighted graph $G = (V, E, w)$, find a spanning tree with minimum total edge weight.

A **spanning tree** is a tree connecting all $|V|$ vertices using exactly $|V| - 1$ edges.

Properties

- Every connected graph has a spanning tree.
- There can be multiple MSTs if edges have equal weights.
- The MST is unique if all edge weights are distinct.

Greedy Property (Cut Property)

Lemma: Let $S \subset V$ and (u, v) be the minimum-weight edge crossing the cut $(S, V \setminus S)$. Then (u, v) is in some MST.

Proof: If an MST T doesn't contain (u, v) , adding it creates a cycle. Removing any other edge in that cycle from S to $V \setminus S$ (call it (u', v')) gives a spanning tree with total weight $\leq w(T)$ (since $w(u, v) \leq w(u', v')$).

Kruskal's Algorithm

Sort edges by weight and greedily add edges that don't create a cycle:

```
def kruskal(G):
    edges = sorted(G.edges, key=lambda e: e.weight)
    uf = UnionFind(G.vertices)
    mst = []

    for (u, v, w) in edges:
        if uf.find(u) != uf.find(v):
            uf.union(u, v)
            mst.append((u, v, w))

    return mst
```

Complexity:

- Sorting: $O(m \log m)$ where $m = |E|$.
- Union-find operations: $O((m + n)\alpha(n))$ where α is the inverse Ackermann function (nearly constant).

Total: $O(m \log m)$.

Prim's Algorithm

Grow the MST by repeatedly adding the minimum-weight edge from the current tree to a new vertex:

```
def prim(G, start):
    mst = {start}
    edges = [(w, start, v) for v in G.neighbors(start)
              for w in [G.weight(start, v)]]
    heapq.heapify(edges)
    mst_edges = []

    while mst != G.vertices:
        w, u, v = heapq.heappop(edges)
        if v not in mst:
            mst.add(v)
            mst_edges.append((u, v, w))
            for x in G.neighbors(v):
                if x not in mst:
                    heapq.heappush(edges, (G.weight(v, x), v, x))

    return mst_edges
```

Complexity: $O((m + n) \log n)$ with a binary heap, $O(n^2)$ with a Fibonacci heap.

13 Lecture 14, 15, 16: Greedy Algorithms and Matroids

0-1 Knapsack Problem

Problem Statement

Given:

- Capacity B (weight limit).
 - n items, each with weight w_i and profit p_i .
- Maximize: $\sum_i p_i x_i$ subject to $\sum_i w_i x_i \leq B$ and $x_i \in \{0, 1\}$.

Why Greedy Fails

Greedy by profit-to-weight ratio p_i/w_i doesn't always work:

Example:

- Item 1: $w_1 = 6, p_1 = 30$, ratio = 5
- Item 2: $w_2 = 3, p_2 = 14$, ratio = 4.67
- Item 3: $w_3 = 4, p_3 = 16$, ratio = 4
- Capacity $B = 7$
- Greedy: Take item 1 (profit 30, weight 6). Remaining capacity 1; can't take anything else. Total = 30.

- Optimal: Take items 2 and 3 (profit $14 + 16 = 30$, weight $3 + 4 = 7$). Total = 30.

In this case greedy ties optimal, but consider if item 3 had profit 18 instead:

- Greedy: 30
- Optimal: $14 + 18 = 32$

Greedy fails!

Dynamic Programming Solution

Recurrence

Let $K(i, j)$ = maximum profit using items $1 \dots i$ with capacity j .

$$K(i, j) = \begin{cases} K(i-1, j) & \text{if } w_i > j \\ \max\{K(i-1, j), p_i + K(i-1, j - w_i)\} & \text{if } w_i \leq j \end{cases}$$

Base case: $K(0, j) = 0$ for all j .

Algorithm

```
def knapsack(w, p, B):
    n = len(w)
    K = [[0] * (B + 1) for _ in range(n + 1)]

    for i in range(1, n + 1):
        for j in range(B + 1):
            K[i][j] = K[i-1][j] # Don't take item i
            if w[i-1] <= j:
                K[i][j] = max(K[i][j], p[i-1] + K[i-1][j - w[i-1]])

    return K[n][B]
```

Complexity: $O(nB)$ time, $O(nB)$ space (can be reduced to $O(B)$ with careful implementation).

When Greedy Works: Matroids

Definition

A matroid $M = (S, \mathcal{I})$ consists of a finite set S and a family $\mathcal{I}$ of subsets satisfying:

1. **Heredity:** If $A \in \mathcal{I}$ and $B \subseteq A$, then $B \in \mathcal{I}$.
2. **Exchange property:** If $|A| < |B|$ with $A, B \in \mathcal{I}$, then $\exists e \in B \setminus A$ such that $A \cup \{e\} \in \mathcal{I}$.

The exchange property guarantees that all maximal feasible sets have the same size (the **rank** of the matroid).

Greedy Algorithm for Matroids

```
def greedy_matroid(M, weights):
    elements = sorted(M.ground_set, key=lambda e: -weights[e])
    solution = []
    for e in elements:
        if can_add(solution, e, M):
            solution.append(e)
    return solution
```

Theorem: For any matroid, the greedy algorithm produces a maximum-weight feasible set.

Example: Graphic Matroid (MST)

The graphic matroid of graph G has:

- $S = E$ (edge set)
- $\mathcal{I}$ = all forests (acyclic edge sets)

This satisfies the matroid axioms. Greedy on forests with maximum weights gives the maximum spanning forest (MST if connected).

14 Lecture 17, 18: Shortest Paths

Single-Source Shortest Paths

Problem

Given a directed graph $G = (V, E, w)$ with weights $w : E \rightarrow \mathbb{R}$ and a source s , compute $d(v)$ = shortest path distance from s to every vertex v .

Constraints

- **No negative cycles:** If negative cycles exist, shortest paths may not be defined.
- **Paths are directed:** (u, v) may have different weight than (v, u) or may not exist.

Bellman-Ford Algorithm

Works with negative weights and detects negative cycles.

```

def bellman_ford(G, s):
    d = {v: float('inf') for v in G.vertices}
    d[s] = 0

    # Relax all edges n-1 times
    for _ in range(len(G.vertices) - 1):
        for (u, v, w) in G.edges:
            if d[u] + w < d[v]:
                d[v] = d[u] + w

    # Check for negative cycles
    for (u, v, w) in G.edges:
        if d[u] + w < d[v]:
            return "Negative cycle detected"

    return d

```

Complexity: $O(|V| \cdot |E|) = O(nm)$.

Why it works: After i iterations, $d(v)$ contains the shortest path distance from s to v using at most i edges. Since shortest paths have at most $n - 1$ edges, $n - 1$ iterations suffice.

Dijkstra's Algorithm

For non-negative edge weights only. Uses a greedy approach with a priority queue.

```

def dijkstra(G, s):
    d = {v: float('inf') for v in G.vertices}
    d[s] = 0
    pq = [(0, s)] # (distance, vertex)
    visited = set()

    while pq:
        dist, u = heapq.heappop(pq)
        if u in visited:
            continue
        visited.add(u)

        for v, w in G.neighbors_with_weights(u):
            if d[u] + w < d[v]:
                d[v] = d[u] + w
                heapq.heappush(pq, (d[v], v))

    return d

```

Complexity: $O((|V| + |E|) \log |V|)$ with a binary heap.

Why non-negative weights are required: Once we pop a vertex from the priority queue, its distance is final. This is only guaranteed if we can't find a shorter path later (impossible with non-negative weights).

Floyd-Warshall: All-Pairs Shortest Paths

Dynamic Programming Approach

Let D_{ij}^k = shortest path from i to j using only intermediate vertices in $\{1, 2, \dots, k\}$.

$$D_{ij}^k = \min\{D_{ij}^{k-1}, D_{ik}^{k-1} + D_{kj}^{k-1}\}$$

Base case: $D_{ij}^0 = w(i, j)$ (direct edge weight or ∞ if no edge).

```
def floyd_warshall(G):
    n = len(G.vertices)
    D = [[float('inf')] * n for _ in range(n)]

    # Initialize
    for i in range(n):
        D[i][i] = 0
    for (u, v, w) in G.edges:
        D[u][v] = w

    # DP
    for k in range(n):
        for i in range(n):
            for j in range(n):
                D[i][j] = min(D[i][j], D[i][k] + D[k][j])

    return D

Complexity:  $O(n^3)$ .
Space:  $O(n^2)$ .
```

15 Lecture 19, 20: Flow Networks and Bipartite Matching

Maximum Flow Problem

Definitions

- **Flow network:** Directed graph with capacities $c(u, v) \geq 0$ on edges.
- **Flow:** Function $f(u, v)$ satisfying $0 \leq f(u, v) \leq c(u, v)$.
- **Flow conservation:** For all $v \neq s, t$, $\sum_u f(u, v) = \sum_w f(v, w)$.
- **Flow value:** $|f| = \sum_v f(s, v)$ (total flow out of source).

Ford-Fulkerson Method

Repeatedly find augmenting paths and push flow until no more exist.

```
def ford_fulkerson(G, s, t):
    flow = 0
    residual = build_residual_graph(G)

    while has_path(residual, s, t): # BFS or DFS
        path = find_path(residual, s, t)
        bottleneck = min(residual.capacity(u, v) for (u, v) in path)

        for (u, v) in path:
            residual.capacity(u, v) -= bottleneck
            residual.capacity(v, u) += bottleneck # Reverse edge

        flow += bottleneck
```

return flow

Complexity

- If capacities are integers, each augmentation increases flow by ≥ 1 , and max flow is $\leq \sum_v c(s, v)$, giving $O(E \cdot \text{max flow})$ time.
- **Edmonds-Karp**: Using BFS for shortest augmenting path, complexity is $O(VE^2)$.
- **Dinic's algorithm**: $O(V^2E)$.
- **Push-relabel**: $O(V^3)$ or $O(V^2\sqrt{E})$ with optimizations.

Max-Flow Min-Cut Theorem

Theorem: The maximum flow value equals the minimum cut capacity.

A **cut** (S, T) partitions vertices with $s \in S, t \in T$. Its capacity is $c(S, T) = \sum_{u \in S, v \in T} c(u, v)$.

Proof sketch:

1. Any flow value $\leq$ any cut capacity (by flow conservation).
2. At max flow, no augmenting path exists.
3. Vertices reachable from s in residual graph form a cut (S, T) with capacity = flow value.

Bipartite Matching via Max Flow

Given bipartite graph (L, R, E) , find maximum matching (set of edges with no shared vertices).

Reduction:

1. Add source s connected to all L vertices with capacity 1.
2. Set capacity 1 on all edges in E (between L and R).
3. Add sink t connected from all R vertices with capacity 1.
4. Compute max flow from s to t .

Each unit of flow corresponds to a matched edge. Max flow = maximum matching size.

16 Lecture 21-23: NP-Completeness and Approximation

Complexity Classes P and NP

Decision Problems

A **decision problem** answers "yes" or "no" to a given input.

Class P

P = problems solvable in polynomial time by a deterministic algorithm.

Examples:

- Sorting: Verify that a list is sorted in $O(n)$ time.
- Shortest path: Given a graph and vertices s, t , is there a path of length $\leq k$? Dijkstra solves in $O((n + m) \log n)$ time.

Class NP

NP = problems for which a proposed solution can be verified in polynomial time.

Formally: $L \in NP$ iff there exist a polynomial p and a deterministic TM M such that: - If $x \in L$, then $\exists$ certificate c with $|c| \leq p(|x|)$ and $M(x, c) = \text{yes}$. - If $x \notin L$, then $\forall$ certificates c , $M(x, c) = \text{no}$.

Examples:

- SAT: Given Boolean formula, is it satisfiable? Certificate = an assignment. Verify in $O(n)$ time.
- Hamiltonian cycle: Does graph have a Hamiltonian cycle? Certificate = the cycle. Verify in $O(n)$ time.
- 3-COLOR: Can a graph be colored with 3 colors? Certificate = a coloring. Verify in $O(n+m)$ time.

Open Question: P vs NP

- Clearly $P \subseteq NP$ (if solvable in polynomial time, the solution is a certificate).
- Is $P = NP$? Unknown. Widely believed to be NO.
- If $P = NP$, then certificate problems are as easy as verification—arguably the biggest open problem in computer science.

Polynomial-Time Reductions

Definition

We write $L_1 \leq_p L_2$ (" L_1 reduces to L_2 ") if there's a polynomial-time computable function f such that:

$$x \in L_1 \Leftrightarrow f(x) \in L_2$$

Implications

- If $L_1 \leq_p L_2$ and $L_2 \in P$, then $L_1 \in P$.
- If $L_1 \leq_p L_2$ and L_1 is NP-hard, then L_2 is NP-hard.

NP-Completeness

Definition

L is NP-complete if:

1. $L \in NP$
2. Every problem in NP reduces to L (i.e., L is NP-hard).

Importance

- NP-complete problems are among the hardest in NP .
- If any NP-complete problem is in P , then $P = NP$.
- Hundreds of NP-complete problems are known (scheduling, graph coloring, optimization, etc.).

SAT: The First NP-Complete Problem

Definition

Boolean Satisfiability (SAT): Given a Boolean formula Φ in n variables, is there an assignment making Φ true?

Example: $\Phi = (x_1 \vee \neg x_2) \wedge (x_2 \vee x_3) \wedge \neg x_1$

Try assignment $x_1 = F, x_2 = T, x_3 = T$:

$$(F \vee F) \wedge (T \vee T) \wedge T = F \wedge T \wedge T = F$$

Try $x_1 = F, x_2 = T, x_3 = F$:

$$(F \vee F) \wedge (T \vee F) \wedge T = F \wedge T \wedge T = F$$

(Unsatisfiable formula in this case.)

Cook-Levin Theorem

SAT is NP-complete.

Proof idea: Every NP problem can be solved by a nondeterministic TM in polynomial time. The TM's execution can be encoded as a Boolean formula that is satisfiable iff the NP problem has a "yes" answer.

3-SAT

Definition

3-SAT: SAT restricted to formulas in CNF (Conjunctive Normal Form) where each clause has exactly 3 literals.

Example: $(x_1 \vee \neg x_2 \vee x_3) \wedge (x_2 \vee x_3 \vee \neg x_4) \wedge \dots$

Reduction from SAT

Any SAT formula can be converted to 3-CNF by introducing new variables:

- Long clauses are broken into 3-literal clauses with new variables.
- Single or two-literal clauses are padded (e.g., $(x_1 \vee x_1 \vee x_1)$).

Result: 3-SAT is NP-complete.

Other NP-Complete Problems

Clique and Independent Set

- **Clique:** Given graph G and integer k , does G have a clique of size k ?
- **Independent Set:** Does G have an independent set of size k (no edges within)?
- These reduce to each other: C is a clique in G iff C is an independent set in $\overline{G}$ (complement graph).

Reduction from 3-SAT to Clique: Create vertices for each literal occurrence. Connect two vertices iff they are in different clauses and are consistent (not contradictory). A k -clique corresponds to choosing one true literal from each of k clauses, giving a satisfying assignment.

Vertex Cover

Definition: Given graph G and integer k , is there a vertex cover of size $\leq k$ (a set of vertices covering all edges)?

Relationship: A set S is a vertex cover iff $V \setminus S$ is an independent set.

Thus Vertex-Cover and Independent-Set are equivalent under reduction.

Hamiltonian Cycle and TSP

- **Hamiltonian Cycle:** Does graph G have a cycle visiting every vertex exactly once?
- **TSP (decision):** Given complete graph with edge weights and integer B , is there a tour of total length $\leq B$?

Reduction: For Hamiltonian cycle on G , create a metric TSP instance where edge (i, j) has weight 1 if it exists in G , weight 2 otherwise. A Hamiltonian cycle exists iff there's a TSP tour of length exactly n .

Approximation Algorithms

For NP-hard optimization problems, we often settle for good approximate solutions.

Approximation Ratio

For minimization: Algorithm is α -approximate if $\text{SOL} \leq \alpha \cdot \text{OPT}$ where $\alpha \geq 1$.

For maximization: $\text{SOL} \geq (1/\alpha) \cdot \text{OPT}$.

Vertex Cover 2-Approximation

```
def approx_vertex_cover(G):
    cover = []
    edges = G.edges.copy()
    while edges:
        u, v = edges.pop() # Pick any edge
        cover.extend([u, v])
        # Remove all edges incident to u or v
        edges = [(a, b) for a, b in edges if a not in {u, v} and b not in {u, v}]
    return cover
```

Analysis: Let M be the set of edges picked. The approximation has size $2|M|$. Since M is a matching (no shared vertices), $|M| \leq \text{OPT}$. Thus $\text{SOL} = 2|M| \leq 2 \cdot \text{OPT}$.

Metric TSP 2-Approximation

Assume edge weights satisfy the triangle inequality: $w(i, k) \leq w(i, j) + w(j, k)$.

```
def approx_metric_tsp(G):
    # Compute MST
    mst = kruskal(G)

    # Double edges to make Eulerian
    multigraph = double_edges(mst)

    # Find Euler tour
```

```

euler = find_euler_tour(multigraph)

# Shortcut to get Hamiltonian cycle
tour = []
seen = set()
for v in euler:
    if v not in seen:
        tour.append(v)
        seen.add(v)
tour.append(tour[0]) # Close the cycle

return tour

```

Analysis:

- MST weight $\leq \text{OPT}$ (any TSP tour is a spanning tree).
- Doubling: edges have total weight $2 \cdot \text{MST} \leq 2 \cdot \text{OPT}$.
- Euler tour traverses each edge once, so total distance $\leq 2 \cdot \text{OPT}$.
- Shortcutting: By triangle inequality, we don't increase the cost.
- Final tour: $\leq 2 \cdot \text{OPT}$.

For n distinct keys there are $n!$ possible permutations, so any correct decision tree must have at least $n!$ leaves. A binary tree with L leaves has height at least $\lceil \log_2 L \rceil$, so

$$\text{worst-case comparisons} \geq \log_2(n!) = \Omega(n \log n).$$

Average-case lower bound

For an input distribution, the expected number of comparisons equals the expected depth of the leaf reached by that input. The minimum expected depth over all decision trees is still $\Omega(\log(n!))$, so comparison-based sorting has average-case lower bound $\Omega(n \log n)$.

Bucket sort for uniform $[0, 1]$ inputs

Assume n i.i.d. samples from the uniform distribution on $[0, 1]$. Bucket sort:

1. Create n buckets for intervals $[0, 1/n), [1/n, 2/n), \dots$
2. Insert each x_i into bucket $\lfloor nx_i \rfloor$.
3. Sort each bucket (e.g., insertion sort) and concatenate.

Let B_j be bucket j . Then $\mathbb{E}[|B_j|] = 1$. Using indicator variables, $\mathbb{E}[|B_j|^2] = O(1)$, so

$$\mathbb{E} \left[\sum_j |B_j|^2 \right] = O(n).$$

Thus the expected running time is $O(n)$.

Counting sort and range dependence

For integer keys in $\{0, \dots, m-1\}$, counting sort runs in $O(n+m)$ time. If $m = O(n)$ this is linear; if $m \gg n$ the m term dominates.

Radix sort perspective

Radix sort uses stable counting sort on each digit. It can sort numbers in a large range (0 to $m^d - 1$) in $O(d(n + m))$ time by increasing the number of digits while keeping the per-digit range small.

17 Lecture 9

Binomial heaps: structure

A binomial heap is a set of binomial trees that are heap-ordered, with at most one tree of any order. If the heap has n elements, it contains at most $\lfloor \log_2 n \rfloor + 1$ trees.

Merge (union)

We can merge two binomial heaps in $O(\log n)$ time, where n is the total number of elements. The merge is analogous to binary addition on the tree orders.

Insert

Insert an element by creating a single-node binomial heap and merging it with the existing heap. This takes $O(\log n)$ in the worst case.

Decrease-key

To decrease a key, reduce its value and then bubble the node up the tree while the heap order is violated (swap with its parent until the order is restored).

Find-min

If we maintain an explicit pointer to the minimum root, then MIN takes $O(1)$ time. If not, we must scan all roots, which costs $O(\log n)$. The min pointer must be updated when it changes.

Delete and extract-min

To delete an arbitrary node, first decrease its key to $-\infty$, bubble it up to a root, delete that root, and then merge the heap formed by its children with the remaining heap.

EXTRACT-MIN is just deletion of the minimum root followed by merging its children with the rest, and then recomputing the minimum root.

18 Lecture 10

Priority queues and heaps

A min-priority queue supports:

- MIN: $O(1)$ with a min pointer.
- INSERT, DELETE, EXTRACT-MIN: typically $O(\log n)$.

Binary heaps support building a heap in $O(n)$ time, while building a balanced BST on n keys costs $\Theta(n \log n)$.

Binomial trees

Define a family of binomial trees:

- B_0 is a single node.
- B_{i+1} is obtained by linking two B_i trees (make one root a child of the other).

Key properties of B_i :

- Number of nodes: 2^i .
- Height: i .
- Root degree: i .
- Nodes on level j : $\binom{i}{j}$.

Binomial heap representation

For a heap of size n , represent n in binary. Each 1-bit corresponds to a binomial tree of the matching order; there is at most one tree of each order. Roots are stored in a list ordered by increasing order (often a circular list).

Merging two binomial heaps

When merging two binomial heaps, we combine root lists (by order) and link any two trees with the same order. This is identical to adding two binary numbers and handling carries, so the merge takes $O(\log n)$ time.

19 Lecture 11

Amortized analysis for binomial heap insert

Consider an n -bit counter. The cost of incrementing is the number of bit flips. The worst case is n , but the total number of flips over 2^n increments is:

$$\sum_{j=0}^{n-1} \frac{2^n}{2^j} \leq 2^{n+1}.$$

Thus the amortized cost per increment is $O(1)$.

In a binomial heap, inserting an element corresponds to incrementing the binary representation of the heap size. Each carry corresponds to linking two trees of the same order. Therefore, over n insertions, the total number of links is $O(n)$, giving $O(1)$ amortized time per insert.

20 Lecture 12

Potential method for amortized analysis

Let the data structure states be $D_0, D_1, \dots, D_n$. A potential function $\Phi : D_i \rightarrow \mathbb{R}$ assigns a potential to each state. The amortized cost of operation i is

$$\hat{c}_i = c_i + (\Phi_i - \Phi_{i-1}).$$

Summing gives

$$\sum_{i=1}^n \hat{c}_i = \sum_{i=1}^n c_i + \Phi_n - \Phi_0,$$

so the total actual cost is bounded by total amortized cost plus boundary terms.

Example: bit counter

Let Φ be the number of 1-bits. If an increment flips t trailing 1s to 0 and one 0 to 1, then $c_i = t + 1$ and $\Delta\Phi = 1 - t$, so

$$\hat{c}_i = (t + 1) + (1 - t) = 2.$$

Hence 2^n increments cost at most $2(2^n - 1)$ in total, i.e., $O(1)$ amortized.

Disjoint sets (union-find) with union by size

We maintain a partition of a set S into disjoint subsets. Operations:

- **FIND**(x): return the representative of the set containing x .
- **UNION**(S_i, S_j): merge two sets.

If we represent each set as a star (all elements point to the root), **FIND** is $O(1)$. To union, relabel all elements in the smaller set to point to the larger root, costing $\min\{|S_i|, |S_j|\}$.

Each element changes its parent only when its set is the smaller one. Every time it changes, the size of its set at least doubles, so each element changes at most $\lfloor \log_2 n \rfloor$ times. For $n - 1$ unions and m finds, the total time is

$$O(m + n \log n).$$

21 Lecture 13

Minimum spanning tree (MST)

Given an undirected weighted graph $G = (V, E, w)$, a spanning tree connects all vertices with minimum total weight. The number of spanning trees can be large; for the complete graph K_n it is n^{n-2} (Cayley's formula).

DFS yields a spanning tree, but it is not necessarily minimum. We need a greedy criterion that is provably correct.

Kruskal's algorithm (greedy)

Sort edges by nondecreasing weight and scan in that order. Add an edge if it does not create a cycle. Use union-find to test whether endpoints are in the same component.

Time complexity:

- Sorting: $O(m \log m)$.
- Union/find operations: nearly linear (or $O(m + n \log n)$ with union by size).

Other MST algorithms

- **Prim's algorithm**: grows one tree by repeatedly adding the minimum-weight edge crossing the cut from the tree to the remaining vertices.
- **Boruvka's algorithm**: repeatedly add the cheapest outgoing edge from each component.

22 Lecture 14

0–1 knapsack

Given capacity B and items with weights w_i and profits p_i , solve:

$$\max \sum_i p_i x_i \quad \text{s.t.} \quad \sum_i w_i x_i \leq B, \quad x_i \in \{0, 1\}.$$

Greedy by ratio p_i/w_i can fail; the notes include a counterexample where the greedy profit is smaller than the optimal profit.

Branch-and-bound intuition

The decision tree over $x_1, x_2, \dots$ can be pruned using an upper bound, e.g., the fractional knapsack value. This avoids exploring all 2^n leaves.

Maximum weight matching (bipartite graphs)

A *matching* is a set of edges with no shared vertices. The goal is to maximize the total weight of selected edges. A perfect matching covers every vertex. The constraint is:

No two chosen edges share a vertex.

23 Lecture 15

Matroid definition

A matroid is a pair $M = (S, \mathcal{I})$ where S is the ground set and $\mathcal{I}$ is a family of feasible subsets satisfying:

- Heredity: if $A \in \mathcal{I}$ and $B \subseteq A$, then $B \in \mathcal{I}$.
- Exchange property: if $|A| < |B|$ with $A, B \in \mathcal{I}$, then $\exists e \in B \setminus A$ such that $A \cup \{e\} \in \mathcal{I}$.

The exchange property implies the *rank property*: all maximal feasible subsets of any $A \subseteq S$ have the same size.

Greedy optimality

For matroids, the greedy algorithm that repeatedly adds the maximum-weight element that keeps the set feasible produces an optimal maximum-weight feasible subset.

Minimization via maximization

To solve a minimization problem on a matroid, define $w'(e) = w_{\max} - w(e) \geq 0$. Maximizing total w' is equivalent to minimizing the original total weight.

Application: maximum spanning forest

Let S be the set of edges and $\mathcal{I}$ be all forests. This forms a matroid. The greedy algorithm yields a maximum spanning forest (a spanning tree if the graph is connected).

Job scheduling (unit-time, deadlines, penalties)

Each job takes one unit time, has a deadline, and incurs a penalty if missed. Equivalently, we maximize the total penalty of the scheduled (on-time) jobs. A greedy strategy sorts jobs by decreasing penalty and keeps a feasible subset (one that can be scheduled by deadlines).

24 Lecture 16

Job scheduling and exchange property

For the job scheduling problem, the feasible sets (jobs that can meet all deadlines) form a matroid. An exchange argument shows that if $|A| < |B|$ are feasible, then a job from $B \setminus A$ can be inserted into A while maintaining feasibility.

Knapsack approximation idea

Sort items by decreasing ratio p_i/w_i and fill greedily to capacity B as in fractional knapsack. Let items $1..l$ fit fully and take a fraction of item $l+1$ in the fractional solution. Then

$$\max \left\{ \sum_{i=1}^l p_i, p_{l+1} \right\} \geq \frac{1}{2} \cdot (\text{fractional optimum}).$$

Since the fractional optimum upper-bounds the 0–1 optimum, this yields a $1/2$ -approximation.

Dynamic programming for 0–1 knapsack

Let $K(i, j)$ be the optimal profit using items $1..i$ with capacity j .

$$K(i, j) = \begin{cases} K(i-1, j), & w_i > j, \\ \max\{K(i-1, j), p_i + K(i-1, j - w_i)\}, & w_i \leq j. \end{cases}$$

The answer is $K(n, B)$. This runs in $O(nB)$ time and $O(nB)$ space (or $O(B)$ with row reuse).

25 Lecture 17

Single-source shortest paths: problem setup

Given a weighted directed graph $G = (V, E, w)$ and a source s , find $d(v)$, the shortest path distance from s to every vertex. Paths must respect edge directions; negative edge weights are allowed unless stated otherwise.

Relaxation and optimal substructure

For an edge (u, v) , the relaxation step is

$$d(v) \leftarrow \min\{d(v), d(u) + w(u, v)\}.$$

If shortest paths have been correctly computed for all vertices in a prefix of a topological order (DAG case), a single pass of relaxations produces correct distances for the next vertex.

Shortest paths in DAGs

If G is acyclic, compute a topological ordering and relax edges in that order. Because every edge goes forward in the order, each vertex's distance is finalized when it is processed. The algorithm runs in $O(|V| + |E|)$ time.

Bellman–Ford algorithm

Bellman–Ford performs $|V| - 1$ full passes of edge relaxations. It correctly handles negative weights and detects negative cycles:

1. Initialize $d(s) = 0$, $d(v) = \infty$ for $v \neq s$.
2. Repeat $|V| - 1$ times: relax all edges.
3. If any edge can still be relaxed, a negative cycle is reachable from s .

The running time is $O(|V||E|)$.

Dijkstra's algorithm

For nonnegative edge weights, Dijkstra's algorithm repeatedly extracts the unsettled vertex with minimum tentative distance and relaxes its outgoing edges. With a binary heap it runs in $O((|V| + |E|) \log |V|)$ time. The correctness depends on nonnegative weights; otherwise a vertex could be finalized too early.

26 Lecture 18

All-pairs shortest paths (APSP)

The APSP problem asks for shortest path distances between every ordered pair of vertices. It can be solved by running a single-source algorithm from every vertex, but there are faster dynamic programming approaches for dense graphs.

Floyd–Warshall DP

Let D_{ij}^k be the shortest path from i to j using only intermediate vertices from $\{1, 2, \dots, k\}$. The recurrence is

$$D_{ij}^k = \min\{D_{ij}^{k-1}, D_{ik}^{k-1} + D_{kj}^{k-1}\}.$$

Initialize $D_{ij}^0 = w(i, j)$ (or ∞ if no edge). This yields an $O(|V|^3)$ algorithm and can detect negative cycles when $D_{ii}^n < 0$ for some i .

Path reconstruction

Maintain a predecessor matrix π (or next-hop matrix) that records which intermediate vertex improved a distance. After the DP, reconstruct paths by following π pointers.

Johnson's algorithm

For sparse graphs with possible negative edges (but no negative cycles), reweight edges using a potential function $h(v)$ found by adding a new source and running Bellman–Ford:

$$w'(u, v) = w(u, v) + h(u) - h(v) \geq 0.$$

Then run Dijkstra from every vertex on w' , and convert back to original distances. The total time is $O(|V||E| \log |V|)$.

27 Lecture 19

Flow networks

A flow network is a directed graph with a source s , sink t , and capacity function $c : E \rightarrow \mathbb{R}_{\geq 0}$. A flow f satisfies:

- Capacity: $0 \leq f(u, v) \leq c(u, v)$.
- Conservation: for $v \neq s, t$, $\sum_u f(u, v) = \sum_w f(v, w)$.

The value of the flow is $|f| = \sum_v f(s, v) - \sum_v f(v, s)$.

Residual graph and augmenting paths

The residual capacity of edge (u, v) is $c_f(u, v) = c(u, v) - f(u, v)$. The residual graph G_f has edges with positive residual capacity, including reverse edges that allow canceling flow. Any path from s to t in G_f is an augmenting path, and augmenting by the minimum residual capacity increases the flow value.

Ford–Fulkerson method

Repeatedly find an augmenting path and push flow along it until no augmenting path exists. If capacities are integers, the algorithm terminates with an integral maximum flow. The running time depends on how augmenting paths are chosen.

Edmonds–Karp (BFS paths)

Choosing the shortest (fewest-edge) augmenting path via BFS yields Edmonds–Karp, which runs in $O(|V||E|^2)$ time. The key invariant is that the shortest-path distance in the residual graph never decreases.

Max-flow min-cut theorem

A cut (S, T) with $s \in S, t \in T$ has capacity $c(S, T) = \sum_{u \in S, v \in T} c(u, v)$. For any flow f , $|f| \leq c(S, T)$. The theorem states that the maximum flow value equals the minimum cut capacity, and a flow is maximum iff no augmenting path exists.

28 Lecture 20

Bipartite matching via flow

Given a bipartite graph (L, R, E) , create a source connected to all L vertices with capacity 1, connect L to R edges with capacity 1, and connect R to the sink with capacity 1. Any integral max flow corresponds to a maximum matching.

Vertex-disjoint and edge-disjoint paths

Edge-disjoint s - t paths can be modeled by setting unit capacities on edges and computing a max flow. For vertex-disjoint paths, split each vertex v (except s, t) into v_{in} and v_{out} connected by an edge of capacity 1, and redirect incoming/outgoing edges accordingly.

Circulation with demands

Each edge has lower and upper bounds, $\ell(u, v) \leq f(u, v) \leq c(u, v)$. Transform by sending the lower bounds first and adjusting node demands. Add a super source and super sink to satisfy the net demands and test if a feasible circulation exists using a max flow computation.

Min-cut as a certificate

After computing a max flow, the set of vertices reachable from s in the residual graph forms a minimum cut. This provides an explicit certificate for optimality.

29 Lecture 21

Complexity classes P and NP

P is the class of decision problems solvable in polynomial time. NP consists of problems for which a proposed solution can be verified in polynomial time. Equivalently, a problem is in NP if it is solvable in polynomial time by a nondeterministic TM.

Polynomial-time reductions

We say $A \leq_p B$ if there is a polynomial-time computable function f such that $x \in A \Leftrightarrow f(x) \in B$. If A is hard and $A \leq_p B$, then B is at least as hard as A . Reductions preserve membership in NP .

NP-completeness

A problem is NP-complete if it is in NP and every problem in NP reduces to it. To show a problem X is NP-complete, prove $X \in NP$ and give a reduction from a known NP-complete problem (e.g., SAT or 3-SAT) to X .

SAT and 3-SAT

The SAT problem asks if a Boolean formula has a satisfying assignment. 3-SAT restricts clauses to three literals. SAT is NP-complete (Cook–Levin), and 3-SAT remains NP-complete via a polynomial transformation that breaks long clauses into 3-literal clauses.

30 Lecture 22

Clique, independent set, vertex cover

For a graph $G = (V, E)$:

- A clique is a set of vertices with all edges present.
- An independent set has no edges inside it.
- A vertex cover touches every edge.

These are tightly related: a set C is a vertex cover iff $V \setminus C$ is an independent set. Thus *extscVertex – Cover* and *extscIndependent – Set* reduce to each other.

3-SAT to clique reduction

Construct a graph with a vertex for each literal occurrence in each clause. Connect two vertices iff they are from different clauses and are not contradictory. A size- m clique corresponds to choosing one consistent literal from each of the m clauses, giving a satisfying assignment.

Hamiltonian cycle and TSP (decision)

Hamiltonian cycle asks if a graph contains a cycle visiting every vertex once. The decision version of TSP asks if there is a tour of length at most B . Hamiltonian cycle reduces to TSP by assigning edge weights 1 for existing edges and 2 otherwise.

Subset sum and partition

Subset Sum asks if a subset of numbers sums to exactly B . Partition is the special case where B is half the total sum. Both are NP-complete and can be used to show NP-completeness for knapsack decision versions.

31 Lecture 23

Approximation algorithms

For a minimization problem, an algorithm is an α -approximation if it always returns a solution of cost at most α times optimal (with $\alpha \geq 1$). For maximization, the returned value is at least $1/\alpha$ times optimal.

Vertex cover 2-approximation

Repeatedly pick any edge (u, v) , add both endpoints to the cover, and remove all incident edges. The resulting cover has size at most $2 \cdot \text{OPT}$ because each chosen edge is disjoint and must be covered by at least one vertex in any optimum.

Metric TSP 2-approximation

If edge weights satisfy the triangle inequality, compute an MST, double its edges to obtain an Eulerian multigraph, traverse an Euler tour, and shortcut repeated vertices. The tour length is at most $2 \cdot \text{OPT}$.

Set cover greedy bound

Greedy repeatedly selects the set that covers the largest number of uncovered elements. The approximation ratio is $H_n = 1 + 1/2 + \dots + 1/n = O(\log n)$, which is asymptotically tight unless $P = NP$.

Knapsack FPTAS idea

Scale profits so that the maximum profit is polynomially bounded, then run dynamic programming on the scaled profits. This yields a $(1 - \varepsilon)$ -approximation in time polynomial in n and $1/\varepsilon$.

32 Lecture 24

Randomized algorithms: basic notions

Randomized algorithms use random bits to influence execution. A Las Vegas algorithm always returns a correct answer but has random running time; a Monte Carlo algorithm runs within a fixed time bound but may err with small probability.

Randomized quicksort

Choosing the pivot uniformly at random yields expected $O(n \log n)$ comparisons. The expected number of comparisons is

$$\mathbb{E}[C_n] = 2(n+1)H_n - 4n = O(n \log n),$$

and the probability of the worst-case $O(n^2)$ behavior is very small.

Universal hashing

A family $\mathcal{H}$ of hash functions from U to $\{0, \dots, m-1\}$ is universal if $\Pr_{h \in \mathcal{H}}[h(x) = h(y)] \leq 1/m$ for all $x \neq y$. Using a random $h \in \mathcal{H}$ ensures expected $O(1)$ time for hash table operations, independent of the input distribution.

Bloom filters (probabilistic membership)

Bloom filters maintain a bit array and k hash functions. Insert sets k bits; membership queries can have false positives but no false negatives. The false positive rate is approximately $(1 - e^{-kn/m})^k$ for n inserted elements.

33 Lecture 25

Randomized min-cut (Karger)

Repeatedly pick a random edge and contract its endpoints until only two super-nodes remain. The remaining cut is a candidate minimum cut. A single run succeeds with probability at least $2/(n(n-1))$, so repeating $O(n^2 \log n)$ times yields high success probability.

Amplification and boosting success

If an algorithm succeeds with probability p , repeating it independently r times gives failure probability at most $(1-p)^r$. For Karger's algorithm, $r = O(n^2 \log n)$ makes the failure probability at most $1/n^c$ for any desired constant c .

Polynomial identity testing (Schwartz–Zippel)

To test if a multivariate polynomial P is identically zero, pick a random point from a large set and evaluate. If P has total degree d , then

$$\Pr[P(x) = 0] \leq d/|S|,$$

so evaluating at random points gives a Monte Carlo test with controllable error.

Randomized primality testing

Miller–Rabin is a Monte Carlo test that never declares a composite as “prime” when certain witnesses are found, and otherwise declares “probably prime” with error probability at most 4^{-k} after k random bases.